

LECTURE 19

The Normalization Process: 1NF, 2NF, 3NF

One running example, carried through every stage by hand

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

A Repeatable Procedure, Applied Start to Finish

- 1 First Normal Form (1NF) — atomic values, no repeating groups
- 2 Second Normal Form (2NF) — no partial dependency on a composite key
- 3 Third Normal Form (3NF) — no transitive dependency
- 4 One running example, converted step by step: UNF $\rightarrow$ 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF
- 5 A second, shorter worked example to reinforce the pattern
- 6 Testing for normal forms — the checklist to apply to any relation

An Unnormalized Order Table (UNF)

One row per order, with product lines packed into a repeating group inside the row:

orderNo	custNo	custName	{ productNo, productName, unitPrice, qty }
O100	C001	Ann Beech	{ (P01,Widget,9.99,3), (P02,Gadget,24.99,1) }
O101	C002	Tom Kelly	{ (P01,Widget,9.99,2) }
O102	C001	Ann Beech	{ (P03,Sprocket,4.50,10) }

WARNING

Order O100's single row holds TWO product lines inside one cell — exactly the “repeating group” Lecture 5 ruled out. This is not yet a valid relation at all — it is Unnormalized Form (UNF), the raw starting point.

```
orderNo → orderDate, custNo      ·      custNo → custName, custAddress
productNo → productName, unitPrice ·      { orderNo, productNo } → qty
```

STAGE 1

First Normal Form (1NF)

NOTE

A relation is in 1NF if every attribute holds a single, atomic value — no repeating groups, no nested tables. This is simply Lecture 5's definition of a valid relation, restated as the first checkpoint.

Flatten: give each product line its own row, repeating order-level attributes. A single orderNo no longer identifies one row — the primary key grows composite: {orderNo, productNo}.

Order (orderNo, productNo, orderDate, custNo, custName, custAddress, productName, unitPrice, qty) — 1NF

orderNo	productNo	custNo	custName	productName	unitPrice	qty
O100	P01	C001	Ann Beech	Widget	9.99	3
O100	P02	C001	Ann Beech	Gadget	24.99	1
O101	P01	C002	Tom Kelly	Widget	9.99	2
O102	P03	C001	Ann Beech	Sprocket	4.50	10

1NF fixed the STRUCTURAL problem. It did nothing about the redundancy problem — orderDate/custName/custAddress repeat across O100's two rows; productName/unitPrice for Widget repeat across O100 and O101.

Second Normal Form (2NF): Testing for Partial Dependencies

NOTE

2NF: already in 1NF, and every non-key attribute is fully functionally dependent on the WHOLE primary key — not only part of it. A partial dependency: a non-key attribute depends on a proper subset of a composite key.

Key = {orderNo, productNo}. Check each non-key attribute against both halves:

Non-key attribute	Depends on orderNo alone?	Depends on productNo alone?	Full key?
orderDate	Yes	No	Partial
custNo	Yes	No	Partial
custName / custAddress	Yes (via custNo)	No	Partial
productName	No	Yes	Partial
unitPrice	No	Yes	Partial
qty	No	No	Full — needs BOTH

Six of seven non-key attributes are partially dependent — only qty needs the whole key. This relation VIOLATES 2NF.

Converting to 2NF

Every partially-dependent attribute moves to a relation keyed by whichever part of the composite key determines it:

Order (orderNo, orderDate, custNo, custName, custAddress)

Product (productNo, productName, unitPrice)

OrderLine (orderNo, productNo, qty)

orderNo	orderDate	custNo	productNo	productName	unitPrice	orderNo	productNo	qty
O100	2026-08-01	C001	P01	Widget	9.99	O100	P01	3
O101	2026-08-02	C002	P02	Gadget	24.99	O100	P02	1
O102	2026-08-03	C001	P03	Sprocket	4.50	O101	P01	2
						O102	P03	10

TIP

Product and Order now have single-attribute keys — partial dependency is structurally impossible. OrderLine's only non-key attribute (qty) genuinely needs the entire composite key. Widget's price no longer repeats per order.

But Order still has a problem: does custName really depend on orderNo, or on custNo, which merely happens to be reachable from orderNo? That's exactly what 3NF answers.

Third Normal Form (3NF): Transitive Dependency

NOTE

3NF: already in 2NF, and no non-key attribute is transitively dependent on the primary key. Transitive dependency: Z depends on non-key Y, and Y depends on key X — $X \rightarrow Y \rightarrow Z$, but Z doesn't depend on X directly.

Check Order(orderNo, orderDate, custNo, custName, custAddress), key orderNo: $\text{custNo} \rightarrow \text{custName}, \text{custAddress}$ — but custNo is itself a NON-KEY attribute here. So custName/custAddress depend on orderNo only transitively: $\text{orderNo} \rightarrow \text{custNo} \rightarrow \text{custName}$. VIOLATES 3NF.

Customer (custNo, custName, custAddress)

custNo	custName	custAddress
C001	Ann Beech	12 Elm St, Lahore
C002	Tom Kelly	5 Oak Ave, Karachi

Order (orderNo, orderDate, custNo)

orderNo	orderDate	custNo
O100	2026-08-01	C001
O101	2026-08-02	C002
O102	2026-08-03	C001

Split off the transitively-dependent attributes into Customer, keyed by custNo, leaving custNo behind in Order as a foreign key. Product and OrderLine carry over unchanged.

Final 3NF Schema — Verified BCNF Too

```
Customer  (custNo, custName, custAddress)
Product   (productNo, productName, unitPrice)
Order     (orderNo, orderDate, custNo)
           Foreign Key custNo references Customer(custNo)
OrderLine (orderNo, productNo, qty)
           FK orderNo → Order, FK productNo → Product
```

UNF

One row per order, product lines repeat inside it

1NF

Flattened — one row per (order, product); key grows composite

2NF

Split off attributes depending on only half the composite key

3NF

Split off attributes depending on a non-key attribute, not the key

Every Anomaly Gone — At No Cost

- Product accepts new, unordered product P04 — insertion anomaly gone
- Deleting O101 leaves Customer row C002 intact — deletion anomaly gone
- Widget's price lives in exactly one row of Product — modification anomaly gone

`Customer ⋈ Order ⋈ OrderLine ⋈ Product` reconstructs the original flattened 1NF table exactly

Every join attribute (custNo, orderNo, productNo) is the primary key of the relation on the “one” side of the join — which is exactly why this decomposition is lossless.

Enrollment: Confirming the Pattern Generalizes

Enrollment (studentNo, courseNo, studentName, courseName, instructorNo, instructorOffice, grade) — 1NF

studentNo	courseNo	studentName	courseName	instructorNo	instructorOffice	grade
S1	CS201	Bilal Khan	Databases	I01	Room 214	A
S1	CS305	Bilal Khan	Networks	I02	Room 108	B+
S2	CS201	Sara Malik	Databases	I01	Room 214	A-

FDs: studentNo → studentName; courseNo → courseName, instructorNo; instructorNo → instructorOffice; {studentNo, courseNo} → grade

- 2NF check vs {studentNo, courseNo}: studentName partial (needs studentNo only); courseName & instructorNo partial (need courseNo only); instructorOffice reachable from courseNo alone — partial; only grade needs the full key. VIOLATES 2NF.
- 3NF check on new Course(courseNo, courseName, instructorNo, instructorOffice): instructorOffice depends on instructorNo, itself non-key — transitive. VIOLATES 3NF.

```
Student(studentNo, studentName)  Instructor(instructorNo, instructorOffice)  Course(courseNo, courseName, instructorNo)
                                Enrollment(studentNo, courseNo, grade)
```

Testing Any Relation for Normal Forms

1. Is every attribute value atomic?

No repeating groups, no multi-valued cells. No → not even 1NF; flatten first.



2. Is the key composite? Does every non-key need the WHOLE key?

Single-attribute key → automatically 2NF. Composite → check each attribute against each proper subset.



3. Does every non-key attribute depend ONLY on the key?

If $X(\text{key}) \rightarrow Y(\text{non-key}) \rightarrow Z(\text{non-key})$, Z is transitively dependent → violates 3NF.

WARNING

Partial vs. transitive — the most common exam mix-up: partial is about a composite key (some attribute depends on only part of it). Transitive can happen even with a single-attribute key — the culprit is a non-key attribute standing in the way. If the key is single-attribute, only check for transitive dependency.

KEY TAKEAWAYS

Key Takeaways

- 1 1NF: every attribute value is atomic — fixed by flattening, which often forces the key to become composite
- 2 2NF: every non-key attribute depends on the WHOLE composite key, not part of it — relevant only for composite keys
- 3 3NF: every non-key attribute depends DIRECTLY on the key, never transitively through another non-key attribute
- 4 The running Order example went UNF → 1NF → 2NF → 3NF, ending at Customer, Product, Order, OrderLine — zero redundancy, verified lossless
- 5 Testing any relation: atomicity, then partial dependency (composite keys only), then transitive dependency — in that order
- 6 3NF eliminates most redundancy, but not all — Lecture 20 introduces BCNF, a stricter test